# Guide VHDL DMA

## PSESI M1

Année 2025-2026

## Sommaire


Rappels et contexte ..... 2
- Construction du DMA ..... 2
- Cycle adresse et cycle data sur le bus ..... 3
- Signal vs Registre ..... 3
- Block & Guarded ..... 3

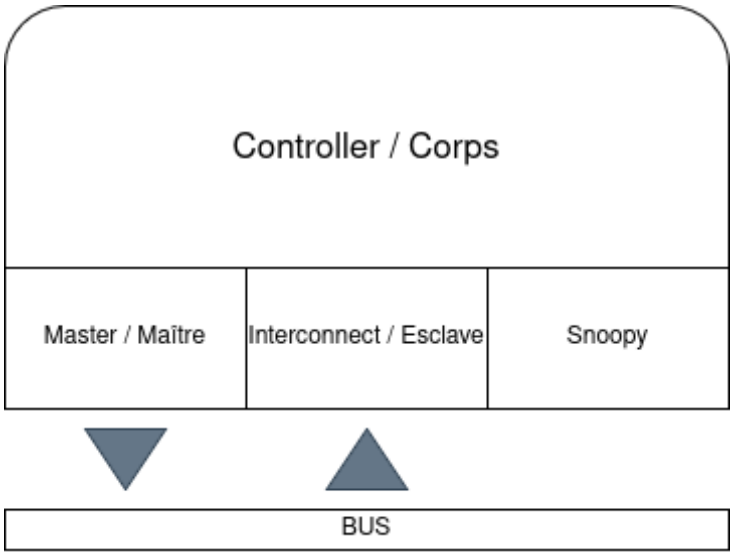
Sections du code ..... 4
- Ports ..... 4
- Architecture ..... 4
  - Signals ..... 4
  - Constants, offsets and masks, zero and one ..... 4
  - Check power supplies ..... 4
  - Internal clock ..... 4
  - Interconnect Controller ..... 4
  - Snoopy: Catching the transfer ..... 5
  - Snoopy: Address comparison ..... 5
  - Interconnect Controller registers ..... 5
  - Snoopy registers ..... 5
  - Assign Outputs - Interconnect Controller ..... 6

Schéma global ..... 6

# Rappels et contexte

## Construction du DMA

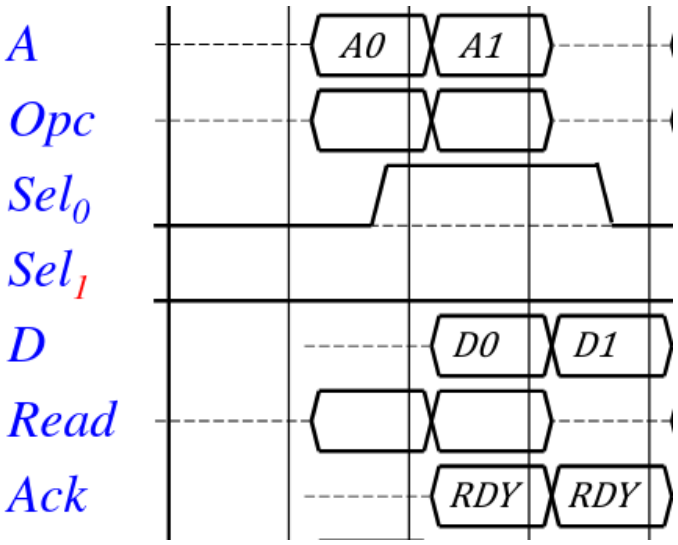
Notre DMA est composé de 4 sous-parties:




- **Interconnect / Esclave**: L'Esclave est exclusivement sollicité lors de la phase de démarrage du SoC. Il réceptionne l'adresse de la racine qui lui est envoyé sur le bus, puis le transmet au Corps et au Snoopy.
- **Snoopy**: Le Snoopy assure une veille constante sur le bus afin de détecter d'éventuelles requêtes modifiant soit la liste des tâches Pending, soit les locks des listes Pending et Done. Dès qu'une requête est identifiée, il notifie le Corps de la nouvelle valeur des ces champs.
- **Master / Maître**: Afin de traiter les requêtes, le Corps doit effectuer diverses opérations de lecture et d'écriture en mémoire. Il délègue ces tâches au Maître, qui reçoit les commandes du Corps et initie les transactions correspondantes sur le bus. Le Maître renvoie ensuite les données nécessaires au Corps pour la poursuite du traitement.
- **Controller / Corps**: Le Corps agit à la fois comme récepteur et initiateur au sein de la DMA. C'est lui qui commande à la partie Maître quelles transactions effectuer sur le bus pour pouvoir récupérer les données d'une tâche et modifier son statut selon son état d'avancement. Le rôle principal du Corps consiste à orchestrer la gestion des transferts entre les données sources et les données de destination.

## Cycle adresse et cycle data sur le bus

Une transaction sur le bus est composée de plusieurs cycles.




Pour notre DMA, nous devons prendre en compte ceci:
Le maître envoie l'adresse de la donnée concernée par la transaction lors d'un cycle. Le data est envoyé qu'au cycle suivant.
Du coup, si un composant sur le bus veut connaître ce que contient la transaction, il lui est impossible de récupérer l'adresse et le data de cette transaction en même temps. Après avoir obtenu l'adresse, il faut attendre un cycle pour pouvoir obtenir le data.

Dans notre SoC, nous pouvons savoir que le bus est au cycle adresse grâce au signal PI_ACYC.
Nous savons aussi lorsque nous sommes au cycle data grâce au signal PI_ACK. Lorsqu'il est positif (READY, READY-MORE ou UNCACHABLE), cela veut dire que l'esclave envoie le data au maître sur le bus.

## Signal vs Registre

Un signal est comme un fil électrique qui relie deux composants ou deux registres. On peut le mettre à 0 ou à 1 selon une condition.

Un registre permet de sauvegarder la valeur d'un signal pour le cycle suivant.

Tous les registres sont mis à 0 lors du reset.

## Block & Guarded

Cela a le même effet que `process(clk)`.

# Sections du code

## Ports

Liste des signaux que le DMA reçoit et envoie. Il y a les signaux CK et RESET qu'on retrouve dans chaque composant synchrone, ainsi que VDD et VSS qui fournissent l'alimentation électrique au DMA. Nous n'utilisons pas la librairie IEEE. Les types que nous utilisons sont bit, mux_bit et mux_vector.

## Architecture

Cette section contient toute la structure interne du DMA. Bien qu'elle soit composée de 4 sous-parties, nous les écrivons toutes dans le même fichier car la complexité reste relativement simple. Séparer chaque partie dans des modules différents ajouterai aussi trop de fils.

### Signals

Liste des signaux internes du DMA. Leur nom est défini par un code:

- Description du signal sur 6 lettres
- Un tiret du bas "_"
- Une lettre pour désigner si c'est un signal (**S**) ou un registre (**R**)
- Une lettre pour désigner à quelle sous-partie appartient le signal (qui l'écrit):
  - **M** aster
  - **S** noopy
  - **I** nterconnect (Esclave)
  - **C** ontroller (Corps)
  - **X** Utilisé pour les reset

### Constants, offsets and masks, zero and one

Liste de variables qui facilitent l'écriture du code. Un peu comme des `#define` en C.

### Check power supplies

Vérification de l'alimentation. VDD a un certain niveau de voltage (généralement 5V ou 12V) et représente la quantité de volts pour le niveau logique 1. VSS est connecté au sol (GND) et représente le niveau de volts pour le niveau logique 0.

### Internal clock

Horloge interne du DMA. Dans notre cas, elle est a la même fréquence que l'horloge du SoC.

### Interconnect Controller

Signaux internes de la partie esclave:

- BASVLD_SI: Signal à 1 quand l'adresse contenue dans le registre BASADR_RI est valide, c'est à dire que l'adresse dans ce registre est bien celle de la racine. Nous savons qu'elle est valide car:
  - Le DMA est selectionné comme cible de la transaction
  - La transaction est une transaction d'écriture
  - La transaction est un transfert d'un mot

  Lorsque ces trois conditions sont réunis, cela veut dire que le CPU, en tant que maître, a initié une transaction afin d'écrire l'adresse de la structure racine dans la DMA.
- ACKRDY_SI: Signal à 1 quand on est prêts à répondre au maître. *Je ne comprends pas la logique derrière ce signal.* Mais à priori, ici, on est prêt à répondre lorsque la transaction sur le bus est une écriture d'un mot.

**Snoopy: Catching the transfer**

Logique des signaux interne permettant de savoir quand récupérer une adresse d'une transaction et quand récupérer le data d'une transaction.
*Conseil: suivre cette partie avec le schéma global à la fin du guide pour mieux comprendre.*

- VLDADR_SS: Signal à 1 lorsqu'il le transfert que le Snoopy écoute est une écriture d'un mot, et qu'on est au cycle d'adresse. Dans une autre section plus bas, sa valeur est enregistrée dans le registre VLDADR_RS.
- DATRDY_SS: Signal à 1 lorsque l'esclave de la transaction que le Snoopy est en train d'écouter à répondu au maître avec un PI_ACK positif (voir la partie PiBus du rappel).
- VLDDAT_SS: Signal à 1 lorsque DATRDY_SS est à 1 et lorsqu'au cycle d'avant, VLDADR_SS était à 1 (on le sait grâce au registre VLDADR_RS).

**Snoopy: Address comparison**

Permet de mettre la valeur qui est en train d'être écrite dans "Pending", "Pending Lock" ou "Done Lock". En gros, nous regardons si l'adresse que nous avons récupéré du bus correspond à un de ces trois champs, puis nous écrivons le data récupéré dans les signaux connectés aux registres partagés avec le Controller/Corps. Cette comparaison est faite à l'aide des signaux suivants:

- CPBAS_SS: Signal qui compare l'adresse observée et l'adresse d'un des champs.
- EQBAS_SS: Signal à 1 lorsque CPBAS_SS est à 0(que les adresses sont identiques).
- MCHBAS_SS: Signal à 1 si l'adresse de la racine est correct, si les adresses sont identiques et si

**Interconnect Controller registers**

Registres de la partie esclave:

- SEL_RI: Registre mis à 1 que quand le bus est au cycle d'adresse et que PI_SEL est à 1 (c.à.d que la cible de la transacion est le DMA).
- WDU_RI: Registre mis à 1 que quand le bus est au cycle d'adresse et que la transaction est la lecture ou l'écriture d'un mot.
- WRT_RI: Registre mis à 1 que quand le bus est au cycle d'adresse et que la transaction est une écriture.
- BASADR_RS: Registre contenant l'adresse envoyée par le maître
- BASVLD_RI: Registre de validité qui confirme que c'est bien l'adresse de la structure racine (grâce à BASVLD_SI).

**Snoopy registers**

Registres de la partie Snoopy:
*Conseil: suivre cette partie avec le schéma global à la fin du guide pour mieux comprendre.*

- VLDADR_RS: Registre mis à 1 lorsque le transfert que le Snoopy écoute est une écriture d'un mot, et qu'on est au cycle d'adresse. On récupère en fait la valeur du signal VLDADR_SS pour le garder un cycle de plus.
- VLDDAT_RS: Registre à 1 lorsque DATRDY_SS est à 1 et lorsqu'au cycle d'avant, VLDADR_SS était à 1 (on le sait grâce au registre VLDADR_RS). On récupère en fait la valeur du signal VLDDAT_SS.
- ADRADR_RS: Registre qui récupère l'adresse de la transaction que le Snoopy est en train d'écouter. Il la récupère quand le bus est au cycle adresse.

- ADRDAT_RS: Registre qui récupère le contenu de ADRADR_RS. Cela marche comme une pipeline. Ça permet d'avoir en même temps l'adresse et le data d'une transaction dans le Snoopy.
- DATDAT_RS: Registre qui récupère le data de la transaction que le Snoopy est en train d'écouter. Il la récupère quand le bus est au cycle data.
- PNDLCK_RS: Registre en commun avec le Controller/Corps qui contient la valeur du lock de la liste Pending.
- DONLCK_RS: Registre en commun avec le Controller/Corps qui contient la valeur du lock de la liste Done.
- PNDADR_RS: Registre en commun avec le Controller/Corps qui contient la valeur du premier élément de la liste Pending.

**Assign Outputs - Interconnect Controller**
Les sorties de la partie esclave. En gros, la réponse de l'esclave au maître qui place la structure racine.

PI_ACK: Actif que quand SEL_I est à 1 (c.à.d que le DMA est sélectionné comme esclave). Envoie la valeur READY sur le bus si le signal ACKRDY est à 1, sinon il envoie ERROR.

# Schéma global
Je n'ai pas encore dessiné la logique pour les registres VLDDAT_RS et VLDADR_RS. En gros c'est ça:
- VLDADR_RS est à 1 lorsque PI_ACYC et PI_READ sont à 1
- VLDDAT_RS est à 1 lorsque VLDADR_RS est à 1 et que PI_ACK est à 1

Lorsque ces 2 registres sont à 1, alors on peut écrire dans les registres partagés entre le Snoopy et le Controller / Corps.

